



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ –
МСХА имени К. А. ТИМИРЯЗЕВА
(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления
Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии
Направленность:
Технологии хранения и управления данными

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и реализация структур данных для хранения истории диалога в LMM-приложениях».

Выполнил:
Студент 1 курса группы Д-Э-22-25
Красноперова Т. А.

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

Оценка _____

Дата защиты _____

Москва 2026

МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА
РОССИЙСКОЙ ФЕДЕРАЦИИ
Российский государственный аграрный университет – МСХА
имени К. А. Тимирязева

Институт экономики и управления АПК
Кафедра статистики и кибернетики

**ЗАДАНИЕ
НА КУРСОВОЙ ПРОЕКТ (КП)**

Обучающийся Красноперова Татьяна Алексеевна
Тема КП Проектирование и реализация структур данных для хранения истории диалога
в LMM приложениях

Исходные данные к работе _____

Перечень подлежащих разработке в работе вопросов:

Перечень дополнительного материала _____

Дата выдачи задания _____ «_____» _____ 202__ г.

Руководитель (подпись, ФИО) _____

Задание принял к исполнению (подпись обучающегося) _____

«_____» _____ 202__ г.

РЕЦЕНЗИЯ

на курсовой проект обучающегося
Федерального государственного бюджетного образовательного учреждения высшего
образования
«Российский государственный аграрный университет – МСХА имени К. А. Тимирязева»

Обучающийся _____

Учебная дисциплина _____

Тема курсового проекта _____

Полнота раскрытия темы: _____

Оформление: _____

Замечания: _____

Курсовой проект отвечает предъявляемым к ней требованиям и
заслуживает _____ оценки.

(отличной, хорошей, удовлетворительной, не удовлетворительной)

Рецензент _____

(фамилия, имя, отчество, уч.степень, уч.звание, должность, место работы)

Дата: «_____» _____ 20__ г.

Подпись: _____

Аннотация

Курсовой проект посвящен проектированию и реализации структур данных для хранения истории диалога в LMM-приложениях. В современных диалоговых системах важную роль играет сохранение переписки между пользователем и программой, так как история сообщений позволяет просматривать предыдущие запросы, находить нужную информацию и управлять содержимым диалога.

В работе рассматриваются основные принципы хранения истории диалога в программном приложении. В качестве основной структуры данных используется список, в котором последовательно сохраняются сообщения пользователя и ответы ассистента. Такой подход является простым, наглядным и удобным для реализации учебного проекта. Каждый элемент истории содержит текст сообщения и относится к определённому участнику диалога: пользователю или ассистенту.

В практической части был разработан программный прототип приложения для работы с историей диалога. Программа позволяет вводить сообщения, сохранять их в истории, отображать переписку в отдельном поле, выполнять поиск по введённому слову, выделять найденный фрагмент светло-жёлтым цветом, а также удалять последнее сообщение из истории диалога. Дополнительно был оформлен пользовательский интерфейс с полем истории, кнопками управления и визуальным фоном, что делает приложение более удобным для использования.

Реализация приложения выполнена с применением объектно-ориентированного подхода и стандартных средств языка программирования. В ходе работы были разработаны основные алгоритмы добавления, поиска, отображения и удаления сообщений из истории. Полученное приложение может использоваться как учебная модель для понимания того, каким образом в LMM-приложениях организуется хранение и обработка диалогового контекста.

В заключительной части работы приводятся выводы о выполненной разработке и рассматриваются возможные направления дальнейшего улучшения приложения, такие как сохранение истории в файл, подключение базы данных, поддержка нескольких диалогов и расширение возможностей поиска.

В работе содержится 5 рисунков, 1 таблица, список литературы состоит из 15 источников, общий объем работы составляет 39 страниц.

Содержание

Введение	6
ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	8
1.1. LMM-приложения и хранение истории диалога	8
1.2. Структуры данных для хранения сообщений	9
1.3. Представление сообщения в истории диалога	10
1.4. Операции над историей диалога	11
1.5. Выбор структуры данных для разрабатываемого приложения	14
1.6. Преимущества и ограничения выбранного подхода	15
1.7. Практическое применение истории диалога	17
ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ	19
2.1. Описание структуры приложения	19
2.1.1. Архитектурный подход и выбор технологий	19
2.1.2. Проектирование основных элементов приложения	19
2.1.3. Схема работы программных компонентов	22
2.2. Алгоритмы обработки данных	23
2.2.1. Общий алгоритм работы приложения	23
2.2.2. Алгоритм добавления сообщения в историю диалога	24
2.3. Описание пользовательской части приложения	25
2.3.1. Построение внешнего вида приложения	25
2.3.2. Последовательность работы пользователя с интерфейсом	27
2.4. Описание контрольного примера	29
2.4.1. Характеристика тестового набора сообщений	29
Заключение	32
Библиографический список	33
Приложение А. Код программы	35

Введение

В настоящее время диалоговые приложения занимают важное место в сфере информационных технологий. Они используются в обучении, справочных системах, технической поддержке, поиске информации, а также в приложениях, связанных с искусственным интеллектом. Особое значение имеют LMM-приложения, то есть программные решения, использующие большие мультимодальные модели [12]. Такие системы способны принимать запросы пользователя, формировать ответы и поддерживать взаимодействие в формате диалога.

Одной из важных частей любого диалогового приложения является история сообщений. Она представляет собой последовательность запросов пользователя и ответов программы, которые сохраняются во время работы приложения. Благодаря истории диалога пользователь может просматривать предыдущие сообщения, находить нужную информацию, возвращаться к уже полученным ответам и при необходимости удалять ненужные элементы переписки. Поэтому при разработке даже простого чат-приложения необходимо заранее продумать способ хранения и обработки сообщений [1].

Актуальность темы курсового проекта связана с тем, что история диалога является основой удобного взаимодействия пользователя с интеллектуальной системой. В реальных LMM-приложениях история используется для сохранения контекста, анализа предыдущих запросов и формирования более точных ответов. Даже если разрабатываемое приложение не подключено напрямую к большой языковой модели, сам принцип хранения сообщений остаётся таким же: каждое сообщение должно быть сохранено, отображено в правильном порядке, найдено по запросу или удалено при необходимости.

В рамках данной курсовой работы рассматривается проектирование и реализация простого программного приложения для хранения истории диалога. Основной структурой данных выбран список, так как он позволяет последовательно добавлять сообщения и сохранять порядок переписки [2]. Такой подход является понятным, наглядным и подходит для учебного проекта, поскольку не требует использования сложных баз данных или серверной части. При этом список позволяет реализовать основные операции, необходимые для работы с историей: добавление сообщений, отображение переписки, поиск по ключевому слову и удаление последних элементов.

Целью курсового проекта является разработка приложения для хранения и обработки истории диалога в LMM-приложениях с использованием простых структур данных. Для достижения данной цели в работе рассматривается назначение истории диалога, изучаются подходящие структуры данных, проектируется логика работы программы и реализуются основные

функции взаимодействия с сообщениями. Особое внимание уделяется операциям добавления сообщений пользователя и ассистента, отображению всей истории, поиску нужного слова с выделением найденного фрагмента, а также удалению последнего сообщения.

Объектом исследования является процесс хранения и обработки истории диалога в программных приложениях. Предметом исследования выступают структуры данных и алгоритмы, применяемые для добавления, поиска, отображения и удаления сообщений в истории переписки.

В практической части курсового проекта разрабатывается приложение, предназначенное для организации и обработки истории диалогового взаимодействия. Пользователь получает возможность вводить сообщения, просматривать сохранённую переписку, выполнять поиск по содержимому истории и удалять последнее сообщение из истории, что отражает базовые принципы работы с контекстом в LMM-приложениях. Реализованный механизм поиска позволяет не только находить нужный фрагмент текста, но и визуально выделять его в области истории, за счёт чего повышается удобство восприятия информации. Пользовательский интерфейс приложения оформлен с применением отдельных экранных элементов и визуального фона, что делает программный продукт более целостным, наглядным и удобным для практического использования.

Практическая значимость работы заключается в том, что разработанное приложение может использоваться как основа для простого чат-бота или учебного LMM-приложения. В дальнейшем проект можно доработать, добавив сохранение истории в файл, поддержку нескольких отдельных диалогов, подключение базы данных и расширенные возможности поиска по сообщениям.

ГЛАВА 1. ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

1.1. LMM-приложения и хранение истории диалога

Современные программные приложения всё чаще используют технологии искусственного интеллекта для взаимодействия с пользователем. Одним из наиболее распространённых вариантов такого взаимодействия является диалоговый режим, при котором пользователь вводит текстовый запрос, а программа формирует ответ. Подобный принцип используется в чат-ботах, виртуальных помощниках, справочных системах и LMM-приложениях.

LMM-приложения можно рассматривать как программные решения, которые используют большие мультимодальные модели для обработки информации [12, 13]. Такие модели могут работать не только с текстом, но и с другими типами данных, например изображениями, аудио или документами. Однако даже в более простом варианте основой взаимодействия остаётся текстовый диалог между пользователем и системой.

Для корректной работы диалогового приложения важно сохранять историю общения. История диалога представляет собой последовательность сообщений, расположенных в том порядке, в котором они были отправлены. Обычно она включает сообщения пользователя и ответы приложения. Такая структура позволяет пользователю видеть ход общения, возвращаться к предыдущим сообщениям и лучше понимать общий контекст переписки.

В LMM-приложениях история диалога имеет особое значение, так как она может использоваться для сохранения контекста [12]. Например, если пользователь сначала задал один вопрос, а затем продолжил тему следующим сообщением, система должна иметь возможность учитывать предыдущие реплики. Без хранения истории приложение воспринимало бы каждый запрос отдельно, что снижало бы удобство и качество взаимодействия.

С точки зрения программирования история диалога является набором данных, который необходимо правильно организовать. Каждое сообщение можно представить как отдельный элемент, содержащий текст, автора сообщения и дополнительные сведения [1, 2]. В более сложных системах также могут храниться дата и время отправки, идентификатор диалога, тип сообщения и другие параметры. В рамках данной курсовой работы используется более простой вариант, при котором основное внимание уделяется хранению текста сообщения и его принадлежности к пользователю или ассистенту.

Для хранения истории диалога могут использоваться разные структуры данных. Наиболее простым и удобным вариантом является список, так как он позволяет последовательно добавлять новые элементы и сохранять порядок сообщений [1]. Именно порядок является важ-

ным свойством диалога, потому что нарушение последовательности сообщений делает переписку непонятной для пользователя.

Таким образом, история диалога является важной частью LMM-приложения. Она обеспечивает сохранение контекста, удобный просмотр переписки и выполнение дополнительных операций, таких как поиск и удаление сообщений. Поэтому выбор подходящей структуры данных является одним из основных этапов проектирования подобного приложения.

1.2. Структуры данных для хранения сообщений

При разработке приложения для хранения истории диалога важным этапом является выбор подходящей структуры данных. Структура данных определяет, каким образом сообщения будут размещаться в памяти программы, как они будут добавляться, отображаться, находиться и удаляться. От правильного выбора структуры зависит удобство дальнейшей реализации и скорость работы основных функций приложения [1, 2].

История диалога представляет собой последовательность сообщений, расположенных в определённом порядке. Обычно первое сообщение находится в начале переписки, а новые сообщения добавляются в конец. Поэтому для хранения такой информации хорошо подходят линейные структуры данных, в которых элементы располагаются друг за другом. Наиболее простым примером такой структуры является список.

Список позволяет хранить набор элементов и последовательно добавлять в него новые данные. В рамках разрабатываемого приложения каждый элемент списка представляет собой отдельное сообщение. Оно может содержать текст, а также информацию о том, кто является автором сообщения: пользователь или ассистент. Такой способ хранения является удобным, потому что порядок сообщений сохраняется автоматически, а вся история может быть выведена на экран от первого сообщения к последнему.

Кроме списка, для хранения сообщений могут использоваться и другие структуры данных [1, 4]. Например, очередь применяется в тех случаях, когда необходимо обрабатывать элементы в порядке их поступления. Такой подход подходит для задач, где важно получать самые старые сообщения первыми. Однако в разрабатываемом приложении требуется не только добавлять сообщения, но и просматривать всю историю, выполнять поиск и удалять последние элементы. Поэтому обычный список является более удобным решением.

Также для хранения диалогов может использоваться словарь. В словаре данные хранятся в виде пары «ключ – значение» [2, 4]. Например, ключом может быть номер сообщения, а значением – его текст. Такой подход удобен для быстрого доступа к конкретному сообщению

по идентификатору. Однако для простого учебного приложения использование словаря не является обязательным, так как основная задача заключается в последовательном хранении всей переписки.

В более сложных LMM-приложениях могут применяться древовидные или графовые структуры [1, 2]. Они позволяют хранить не только обычную линейную переписку, но и разные ветви диалога, например если пользователь возвращается к предыдущему вопросу или развивает несколько тем одновременно. Однако такие структуры сложнее в реализации и больше подходят для крупных систем. В рамках данной курсовой работы используется более простой подход, основанный на линейном хранении сообщений.

Таким образом, для реализации истории диалога в учебном приложении наиболее рационально использовать список. Он обеспечивает сохранение порядка сообщений, простое добавление новых элементов, возможность последовательного просмотра истории и выполнение поиска по содержимому. Такой вариант соответствует целям курсового проекта и позволяет реализовать основные функции без усложнения программной архитектуры.

1.3. Представление сообщения в истории диалога

Для правильной организации истории диалога недостаточно выбрать только общую структуру данных. Также необходимо определить, каким образом будет представлено отдельное сообщение внутри программы. Каждое сообщение является самостоятельным элементом истории и должно содержать данные, которые позволяют корректно отобразить его в интерфейсе и использовать при выполнении основных операций.

В простом диалоговом приложении сообщение можно рассматривать как объект, состоящий из нескольких основных частей [6, 7]. В первую очередь оно содержит текст, который был введен пользователем или сформирован программой в качестве ответа. Именно текст является главным содержанием сообщения, так как с ним связаны такие операции, как отображение в истории, поиск по ключевому слову и удаление из переписки.

Кроме текста, важно учитывать автора сообщения. В истории диалога обычно присутствуют как минимум два участника: пользователь и ассистент. Разделение сообщений по автору позволяет сделать переписку более понятной и визуально упорядоченной. Например, сообщения пользователя могут отображаться с одной подписью, а ответы ассистента – с другой. Благодаря этому пользователь сразу понимает, к какой стороне относится каждая реплика.

В более сложных приложениях сообщение может содержать дополнительные данные: дату и время отправки, уникальный номер, тип сообщения, сведения о вложениях или связь

с конкретным диалогом. Однако в рамках данной курсовой работы используется упрощённая модель, так как целью проекта является демонстрация базовых принципов хранения истории диалога. Поэтому основными параметрами сообщения являются его текст и принадлежность к определённому участнику переписки.

С точки зрения программной реализации сообщение удобно представить в виде отдельного класса [6, 9]. Такой подход соответствует объектно-ориентированному программированию и позволяет отделить данные сообщения от общей логики работы приложения. Например, класс сообщения может хранить текст и автора, а сама история диалога будет представлять собой список таких объектов или строковых записей. Это делает код более понятным и упрощает его дальнейшее развитие.

В разрабатываемом приложении сообщения сохраняются последовательно, в том порядке, в котором они появляются в диалоге. Сначала в историю добавляется сообщение пользователя, затем ответ ассистента. Такая пара сообщений отражает один завершённый этап общения.

Особое значение имеет операция поиска по истории диалога. Для её выполнения программа анализирует текст сообщений и проверяет, содержится ли в них введённое пользователем слово или фраза. Если совпадение найдено, приложение перемещает область просмотра к нужному фрагменту и выделяет его цветом. Такая функция делает работу с историей более удобной, особенно если в переписке накопилось большое количество сообщений.

Таким образом, отдельное сообщение является базовой единицей хранения истории диалога. От того, насколько правильно определена его структура, зависит удобство дальнейшей работы приложения. В рамках данного проекта используется простое представление сообщения, которое включает текст и автора, что достаточно для реализации основных функций: добавления, отображения, поиска и удаления элементов истории.

1.4. Операции над историей диалога

После выбора структуры данных и описания отдельного сообщения необходимо рассмотреть основные операции, которые выполняются над историей диалога. В разрабатываемом приложении история не является статичным набором данных. Она постоянно изменяется во время работы программы: в неё добавляются новые сообщения, пользователь просматривает уже сохранённую переписку, выполняет поиск нужных фрагментов и при необходимости удаляет последние элементы.

Первой и самой важной операцией является добавление сообщения [1, 2]. При вводе текста пользователем программа должна принять это значение, проверить его и поместить в

конец истории. После этого в историю также добавляется ответ ассистента. Таким образом, переписка формируется последовательно: сначала сохраняется запрос пользователя, затем ответ приложения. Такой порядок позволяет сохранить логику диалога и отобразить его в понятном виде.

Второй основной операцией является отображение истории. Пользователь должен видеть все сохранённые сообщения в отдельной области интерфейса. При выводе истории важно сохранить правильную последовательность элементов, так как диалог воспринимается именно как цепочка связанных реплик. Если порядок сообщений нарушить, переписка станет неудобной для чтения и потеряет смысловую целостность.

Отдельное значение имеет поиск по истории диалога. С увеличением количества сообщений пользователю становится сложнее вручную находить нужную информацию. Поэтому в приложении реализуется поиск по введённому слову или фразе [2, 10]. Программа просматривает содержимое истории и определяет, встречается ли искомый фрагмент в сохранённых сообщениях. При нахождении совпадения область истории перемещается к нужному месту, а найденное слово выделяется светло-жёлтым цветом. Это делает поиск более наглядным и удобным.

Также в приложении предусмотрена операция удаления последнего сообщения. В диалоговых приложениях одно действие обычно состоит из двух связанных элементов: сообщения пользователя и ответа ассистента. Поэтому удаление только одного последнего сообщения может нарушить логику переписки. Например, если удалить только ответ ассистента, в истории останется запрос пользователя без результата.

Операция очистки истории может использоваться в тех случаях, когда пользователю необходимо полностью удалить текущую переписку и начать новый диалог. При очистке список сообщений освобождается, а поле истории становится пустым. Данная функция является простой, но полезной, так как позволяет быстро сбросить состояние приложения.

При увеличении объёма истории диалога возникает необходимость не только сохранять сообщения последовательно, но и оценивать их размер и смысловую близость. В LMM-приложениях история может разбиваться на отдельные фрагменты, или чанки. Чанк представляет собой часть переписки, состоящую из одного или нескольких сообщений, которые затем могут использоваться для поиска нужного контекста.

Размер сообщения можно оценивать количеством токенов. В упрощённом виде сообщение можно представить как строку:

$$m_i = (id_i, sender_i, text_i, time_i)$$

где m_i – отдельное сообщение истории, id_i – его порядковый номер, $sender_i$ – отправитель сообщения, $text_i$ – текст сообщения, $time_i$ – время добавления сообщения [6].

Если история диалога содержит n сообщений, то её можно представить в виде последовательности:

$$H = \{m_1, m_2, m_3, \dots, m_n\}$$

где H – вся история диалога, $m_1, m_2, \dots, m_n$ – сообщения, расположенные в порядке добавления.

Для ограничения размера передаваемого контекста история может быть разделена на чанки:

$$H = \{C_1, C_2, C_3, \dots, C_k\}$$

где C_j – отдельный чанк истории, k – количество полученных чанков.

Размер чанка можно определить как сумму размеров входящих в него сообщений:

$$|C_j| = \sum_{i=1}^r |m_i|$$

где $|C_j|$ – размер чанка, $|m_i|$ – размер отдельного сообщения, r – количество сообщений внутри чанка.

Чтобы чанк не превышал допустимый размер контекста, используется условие:

$$|C_j| \leq L$$

где L – максимально допустимый размер чанка. В реальных LMM-приложениях такое ограничение связано с тем, что модель может обрабатывать только ограниченный объём входных данных.

Для поиска нужной части истории может использоваться оценка близости между запросом пользователя и чанками истории. Один из распространённых способов – косинусная мера сходства:

$$\text{sim}(q, C_j) = \frac{q \cdot C_j}{\|q\| \cdot \|C_j\|}$$

где q – векторное представление запроса пользователя, C_j – векторное представление чанка истории, $q \cdot C_j$ – скалярное произведение векторов, $|q|$ и $|C_j|$ – длины соответствующих векторов.

Чем ближе значение $\text{sim}(q, C_j)$ к 1, тем больше чанк соответствует запросу пользователя. Наиболее подходящий чанк можно определить по формуле:

$$C^* = \arg \max_{C_j \in H} \text{sim}(q, C_j)$$

где C^* – чанк истории, наиболее близкий к текущему запросу пользователя.

В рамках разрабатываемого приложения используется более простой вариант поиска – по совпадению введённого слова с текстом сообщений. Однако приведённые формулы показывают, каким образом подобный механизм может быть расширен для более сложных LMM-приложений, где история диалога анализируется не только по точному совпадению слов, но и по смысловой близости.

Таким образом, работа с историей диалога включает несколько базовых операций: добавление сообщений, отображение переписки, поиск по содержимому, удаление последнего сообщения и очистку истории. Все эти действия строятся на использовании выбранной структуры данных и позволяют реализовать минимально необходимый функционал для учебного LMM-приложения.

1.5. Выбор структуры данных для разрабатываемого приложения

После рассмотрения возможных вариантов хранения сообщений необходимо определить, какая структура данных будет использоваться в разрабатываемом приложении. Выбор структуры должен соответствовать не только теоретическим требованиям, но и практическим возможностям проекта. Так как курсовая работа ориентирована на создание простого приложения для хранения истории диалога, слишком сложные решения, например базы данных, графовые структуры или серверное хранение, в данном случае не являются обязательными.

Основной задачей приложения является последовательное сохранение сообщений пользователя и ответов ассистента. Это означает, что новые элементы должны добавляться в конец истории, а при просмотре переписки они должны отображаться в том же порядке, в котором были созданы. Для такой задачи наиболее подходящей структурой данных является список [1, 2].

Он позволяет хранить набор элементов, быстро добавлять новые записи и последовательно проходить по всей истории при её выводе на экран [1].

В рамках приложения каждое сообщение добавляется в историю после действия пользователя. Сначала сохраняется введенный пользователем текст, затем формируется и сохраняется ответ ассистента. Таким образом, список содержит не отдельные случайные данные, а упорядоченную последовательность связанных сообщений. Благодаря этому история диалога сохраняет логическую структуру и может быть корректно отображена в поле переписки.

Использование списка также удобно при реализации удаления последнего сообщения. Так как новые элементы добавляются в конец истории, последние два элемента списка соответствуют последнему запросу пользователя и последнему ответу ассистента. При нажатии соответствующей кнопки программа может удалить именно эти два элемента, не затрагивая остальную часть переписки. Такой подход является простым и понятным с точки зрения программной логики.

Для поиска сообщений список также подходит, так как программа может последовательно просмотреть все элементы истории и проверить, содержится ли в них нужное слово или фраза. Если совпадение найдено, приложение перемещает область просмотра к найденному фрагменту и выделяет его цветом. Несмотря на то, что такой поиск не является самым быстрым для очень больших объёмов данных, для учебного приложения и небольших диалогов он является достаточным и удобным в реализации.

Таким образом, в качестве основной структуры данных для разрабатываемого приложения выбран список. Он соответствует характеру истории диалога, так как сохраняет порядок сообщений, поддерживает добавление новых элементов, позволяет выполнять последовательный просмотр, поиск и удаление последних записей. Данный выбор делает программу более простой, наглядной и подходящей для реализации в рамках курсового проекта.

1.6. Преимущества и ограничения выбранного подхода

Для полноценного обоснования разрабатываемого приложения необходимо рассмотреть не только удобство выбранной структуры данных, но и её ограничения. В рамках курсового проекта основой хранения истории диалога является список. Данный подход выбран потому, что он соответствует последовательной природе переписки и позволяет реализовать основные функции приложения без значительного усложнения кода.

Главным преимуществом списка является простота реализации. Сообщения в диалоге появляются одно за другим, поэтому их удобно добавлять в конец общей структуры. Такой

способ хранения хорошо отражает реальный порядок общения между пользователем и ассистентом. При выводе истории программа может последовательно пройти по элементам списка и отобразить сообщения в том же порядке, в котором они были созданы.

Вторым преимуществом является наглядность. При использовании списка легко понять, где находится первое сообщение, где последнее, и какие элементы относятся к последней паре диалога. Это особенно важно для учебного проекта, так как структура данных должна быть не только рабочей, но и понятной при объяснении алгоритма. Благодаря этому проще описать логику добавления, поиска и удаления сообщений в практической части работы.

Также список удобен при реализации функции удаления последних сообщений. Поскольку новые записи добавляются в конец истории, последние элементы списка соответствуют последнему этапу общения.

Ещё одним преимуществом является возможность простого поиска. Программа может последовательно просматривать элементы списка и сравнивать их содержимое с введённым пользователем словом или фразой. Если совпадение найдено, оно выделяется в области истории. Для небольшого приложения такой способ является достаточным, так как объём сообщений обычно невелик и поиск выполняется быстро.

Однако выбранный подход имеет и определённые ограничения. Список не является самым эффективным вариантом для работы с очень большими объёмами данных. Если история диалога будет содержать тысячи или десятки тысяч сообщений, последовательный поиск может занимать больше времени. В таких случаях целесообразно использовать дополнительные структуры данных, например словари, индексы или базу данных.

Также список хранит сообщения только в рамках текущей работы приложения, если отдельно не реализовано сохранение данных в файл или базу данных. Это означает, что после закрытия программы история может быть потеряна. Для учебного проекта данный вариант допустим, но для полноценного LMM-приложения необходимо предусмотреть долговременное хранение переписки.

Кроме того, линейный список плохо подходит для сложных сценариев, где диалог может иметь несколько ветвей. Например, если пользователь возвращается к старому вопросу и развивает другую тему, обычный список всё равно будет хранить все сообщения только в одном порядке. Для таких случаев лучше подходят древовидные или графовые структуры, но их реализация сложнее и выходит за рамки данного проекта.

Таким образом, использование списка является рациональным решением для разрабатываемого приложения. Он позволяет просто и понятно реализовать основные операции с исто-

рией диалога: добавление, отображение, поиск и удаление сообщений. Несмотря на отдельные ограничения, данный подход соответствует целям курсового проекта и подходит для создания учебного программного прототипа.

1.7. Практическое применение истории диалога

История диалога используется во многих программных решениях, где взаимодействие между пользователем и системой происходит в форме сообщений. Она позволяет сохранять ход общения, возвращаться к предыдущим вопросам и ответам, а также выполнять дополнительные операции над перепиской. В LMM-приложениях история особенно важна, так как она помогает поддерживать контекст и делает работу с программой более удобной.

Основные области применения истории диалога:

Чат-боты и виртуальные ассистенты. В таких системах история сообщений используется для сохранения предыдущих запросов пользователя и ответов программы. Это позволяет ассистенту учитывать уже заданные вопросы и не воспринимать каждое новое сообщение отдельно. Например, если пользователь сначала задаёт общий вопрос, а затем уточняет его, приложение может использовать историю для понимания продолжения темы.

Системы технической поддержки. В сервисах поддержки история диалога помогает специалисту или автоматизированной системе видеть всю переписку с пользователем. Благодаря этому можно быстрее понять проблему, проследить, какие действия уже были предложены, и не заставлять пользователя повторять одну и ту же информацию несколько раз.

Образовательные приложения. В обучающих системах история диалога может сохранять вопросы ученика, ответы приложения, объяснения и результаты проверки знаний. Это удобно для повторения материала и анализа того, какие темы уже были изучены. Также история может использоваться для поиска нужного объяснения, если пользователь хочет вернуться к нему позже.

Справочные и консультационные системы. Такие приложения часто используются для получения информации по определённой теме. История диалога позволяет пользователю сохранять последовательность запросов и быстро находить ранее полученные сведения. Это особенно полезно, если пользователь работает с большим количеством информации и обращается к приложению несколько раз за одну сессию.

LMM-приложения с обработкой разных типов данных. В более сложных мультимодальных системах история может включать не только текстовые сообщения, но и сведения об изображениях, документах или других объектах, с которыми взаимодействовал пользователь. В та-

ком случае история становится частью общего контекста работы приложения и помогает связать разные действия пользователя в единую последовательность.

Учебные программные проекты. История диалога также подходит для демонстрации работы базовых структур данных. На её примере можно наглядно показать использование списка, добавление элементов, последовательный просмотр, поиск по содержимому и удаление последних записей. Поэтому выбранная тема хорошо подходит для курсового проекта по программированию.

Таким образом, хранение истории диалога имеет большое практическое значение. Оно применяется как в простых чат-приложениях, так и в более сложных интеллектуальных системах. В рамках данной курсовой работы рассматривается учебная реализация такого механизма, которая показывает основные принципы работы с сообщениями и может быть использована как основа для дальнейшего развития приложения.

ГЛАВА 2. ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1. Описание структуры приложения

2.1.1. Архитектурный подход и выбор технологий

При разработке приложения был выбран простой архитектурный подход, основанный на разделении пользовательского интерфейса и логики работы с историей сообщений. Такой вариант является удобным для учебного проекта, так как позволяет отдельно рассматривать внешний вид программы и внутреннюю обработку данных.

Разрабатываемое приложение представляет собой простую диалоговую систему, в которой пользователь может вводить текстовые сообщения, просматривать историю переписки, выполнять поиск по содержимому и удалять последнее сообщение. Основной задачей приложения является демонстрация того, каким образом структуры данных могут применяться для хранения истории диалога в LMM-приложениях.

В качестве основной структуры данных используется список. Он позволяет сохранять сообщения в том порядке, в котором они были добавлены пользователем или сформированы приложением. Новые элементы помещаются в конец списка, что соответствует естественной последовательности диалога. При отображении истории программа последовательно выводит сохранённые сообщения в область переписки.

Выбор списка обусловлен тем, что он прост в реализации и хорошо подходит для хранения линейной истории сообщений. С его помощью можно выполнить все основные операции, необходимые для приложения: добавление нового сообщения, просмотр истории, поиск по содержимому и удаление последней записи. При этом не требуется подключение сложных внешних средств хранения, например базы данных или серверной части.

Приложение реализуется с использованием стандартных средств языка программирования и объектно-ориентированного подхода [6, 7]. Это позволяет представить отдельные элементы программы в виде самостоятельных объектов и сделать код более понятным. Такой подход также упрощает дальнейшее развитие проекта, например добавление сохранения истории в файл или расширение функций поиска.

2.1.2. Проектирование основных элементов приложения

При проектировании приложения важно не только определить внешний вид интерфейса, но и продумать внутреннюю структуру программы. Для этого основные элементы приложения

разделяются по назначению: одни отвечают за хранение сообщений, другие – за выполнение действий пользователя, третьи – за отображение данных в интерфейсе.

В рамках разрабатываемого приложения можно выделить несколько основных логических компонентов.

Класс `Message` – сообщение диалога.

Данный класс можно рассматривать как базовую единицу истории переписки [6, 9]. Он предназначен для хранения информации об отдельном сообщении, которое добавляется в историю диалога. В более сложных системах сообщение может содержать большое количество параметров, однако в рамках данного учебного приложения используется упрощённая структура.

Основные поля класса `Message`:

`text` – текст сообщения, введённый пользователем или сформированный приложением. Это основное содержимое записи, которое отображается в истории диалога и используется при выполнении поиска.

`sender` – отправитель сообщения. Данное поле позволяет определить, кому принадлежит сообщение: пользователю или ассистенту. Благодаря этому история переписки становится более понятной и логически разделённой.

`time` – время добавления сообщения. Этот параметр может использоваться для отображения момента отправки сообщения, однако в базовой версии приложения он не является обязательным.

Главная задача класса `Message` заключается в том, чтобы хранить данные одного элемента истории. Он не выполняет сложных вычислений, а служит контейнером для информации о сообщении.

Класс `DialogHistory` – история диалога.

Это основной класс, отвечающий за хранение всей переписки. Внутри него располагается список сообщений, куда последовательно добавляются новые элементы. Именно этот класс показывает практическое применение выбранной структуры данных [6, 10].

Основное поле класса `DialogHistory`:

`messages` – список сообщений, в котором хранятся все элементы истории диалога. Новые сообщения добавляются в конец списка, благодаря чему сохраняется правильная последовательность переписки.

Основные методы класса `DialogHistory`:

`addMessage` – метод добавления нового сообщения в историю. Он получает текст сооб-

щения и информацию об отправителе, после чего помещает новую запись в список.

`showHistory` – метод отображения истории. Он последовательно проходит по списку сообщений и выводит их в область истории диалога.

`findMessage` – метод поиска сообщения. Он проверяет содержимое истории и определяет, встречается ли в ней слово или фраза, введённая пользователем.

`deleteLastMessage` – метод удаления последнего сообщения. Он обращается к последнему элементу списка и удаляет его, после чего история обновляется.

`clearHistory` – метод полной очистки истории. Он удаляет все элементы списка и возвращает приложение в исходное состояние.

Класс `MainForm` – основная форма приложения.

Данный класс отвечает за взаимодействие пользователя с программой. В нём располагаются элементы интерфейса: поле ввода, область истории и кнопки управления. Через эту форму пользователь отправляет сообщения, запускает поиск, удаляет последнее сообщение и очищает историю.

Основные элементы формы `MainForm`:

`textBoxInput` – поле ввода сообщения. В него пользователь вводит текст, который затем передаётся в историю.

`historyBox` – область отображения истории диалога. В ней показываются все сохранённые сообщения.

`buttonSend` – кнопка отправки сообщения. При нажатии текст из поля ввода добавляется в историю.

`buttonFind` – кнопка поиска. Она запускает проверку истории по введённому слову или фразе.

`buttonDeleteLast` – кнопка удаления последнего сообщения. Она позволяет убрать последнюю запись из истории без полной очистки переписки.

`buttonClear` – кнопка очистки истории. При её использовании вся переписка удаляется.

Таким образом, внутренняя структура приложения строится на простом разделении обязанностей [7, 15]. Класс `Message` описывает отдельное сообщение, класс `DialogHistory` отвечает за хранение и обработку списка сообщений, а класс `MainForm` обеспечивает взаимодействие пользователя с программой. Такое устройство делает проект понятным, удобным для реализации и подходящим для демонстрации работы структур данных в рамках курсового проекта.

2.1.3. Схема работы программных компонентов

После определения основных классов необходимо описать, как они взаимодействуют между собой во время выполнения программы. Общая логика приложения строится вокруг обработки действий пользователя и изменения содержимого истории диалога.

Работа начинается с основной формы приложения. Пользователь вводит текст сообщения в поле ввода и нажимает кнопку отправки. После этого обработчик события получает введенный текст и проверяет, не является ли он пустым. Если сообщение содержит данные, оно передаётся в компонент, отвечающий за хранение истории, и добавляется в общий список сообщений.

Список истории выступает центральным хранилищем данных. Все новые сообщения помещаются в конец списка, за счёт чего сохраняется порядок переписки. После изменения списка программа обновляет область отображения истории, чтобы пользователь сразу видел добавленную запись.

При выполнении поиска взаимодействие компонентов происходит по похожему принципу. Пользователь вводит искомое слово или фразу и запускает поиск с помощью кнопки. Программа обращается к сохранённой истории и последовательно проверяет содержимое сообщений. Если совпадение найдено, соответствующий фрагмент выделяется в области истории, а пользователь получает визуальный результат поиска.

Удаление последнего сообщения также связано с изменением списка. При нажатии кнопки удаления программа проверяет, есть ли в истории хотя бы один элемент. Если список не пустой, последний элемент удаляется, после чего область отображения снова обновляется. Такой порядок действий позволяет поддерживать актуальное состояние истории без полной очистки диалога.

Очистка истории выполняется через отдельную команду. В этом случае из списка удаляются все сохранённые элементы, а поле истории становится пустым. Данная операция используется для полного сброса текущей переписки.

Таким образом, компоненты приложения работают последовательно: форма получает действие пользователя, обработчик выполняет нужную операцию, список изменяет содержимое истории, а область отображения показывает обновлённый результат. Такая схема делает работу программы понятной и позволяет разделить хранение данных, обработку действий и визуальное представление сообщений.

2.2. Алгоритмы обработки данных

2.2.1. Общий алгоритм работы приложения

Общий алгоритм функционирования приложения описывает последовательность действий, которые выполняются с момента запуска программы до завершения работы пользователя с историей диалога. Он объединяет основные операции приложения: ввод сообщения, сохранение записи, отображение истории, поиск по содержимому, удаление последнего сообщения и очистку переписки [15].

Последовательность шагов для построения общей блок-схемы:

Начало. Запуск программного приложения.

Блок ввода/вывода: отображение главного окна приложения с областью истории диалога, полем ввода сообщения и кнопками управления.

Процесс: инициализация структуры данных для хранения истории. Создаётся список, в который в дальнейшем будут помещаться сообщения пользователя и ответы ассистента.

Блок ввода/вывода: ожидание действия пользователя. Пользователь может ввести сообщение, выполнить поиск, удалить последнее сообщение или очистить историю.

Условие: проверка выбранного действия пользователя.

Процесс: если пользователь нажимает кнопку отправки, программа считывает текст из поля ввода.

Условие: проверка введённого текста на пустое значение. Если поле пустое, добавление сообщения не выполняется.

Процесс: если текст введён, сообщение добавляется в список истории диалога.

Процесс: обновление области истории. Новое сообщение отображается в общей переписке.

Блок ввода/вывода: если пользователь запускает поиск, программа получает искомое слово или фразу.

Процесс: выполнение последовательного просмотра истории и поиск совпадения в сохранённых сообщениях.

Условие: проверка результата поиска. Если совпадение найдено, найденный фрагмент выделяется в области истории.

Процесс: если пользователь нажимает кнопку удаления, программа проверяет наличие сообщений в истории и удаляет последнее сообщение из списка.

Процесс: если пользователь выбирает очистку истории, список сообщений полностью

очищается, а область истории становится пустой.

Блок ввода/вывода: отображение обновлённого состояния истории диалога после выполненной операции.

Условие: проверка необходимости продолжения работы программы. Если пользователь продолжает работу, приложение возвращается к ожиданию следующего действия.

Конец. Завершение работы приложения.

2.2.2. Алгоритм добавления сообщения в историю диалога

Данный алгоритм отвечает за внесение нового сообщения в историю диалога. Он получает текст из поля ввода, проверяет его корректность и добавляет запись в список сообщений. После этого область истории обновляется, чтобы пользователь видел новое состояние переписки.

Последовательность шагов для построения блок-схемы алгоритма:

Начало функции добавления сообщения.

Блок ввода/вывода: получение текста из поля ввода.

Логическое решение: поле ввода пустое?

Да: переход к завершению алгоритма без добавления записи.

Нет: переход к следующему шагу.

Процесс: удаление лишних пробелов в начале и конце введённой строки.

Процесс: формирование новой записи сообщения с указанием автора и текста.

Процесс: добавление сформированной записи в конец списка истории диалога.

Процесс: обновление области отображения истории.

Процесс: очистка поля ввода после добавления сообщения.

Блок ввода/вывода: вывод обновлённой истории диалога на экран.

Конец функции добавления сообщения.

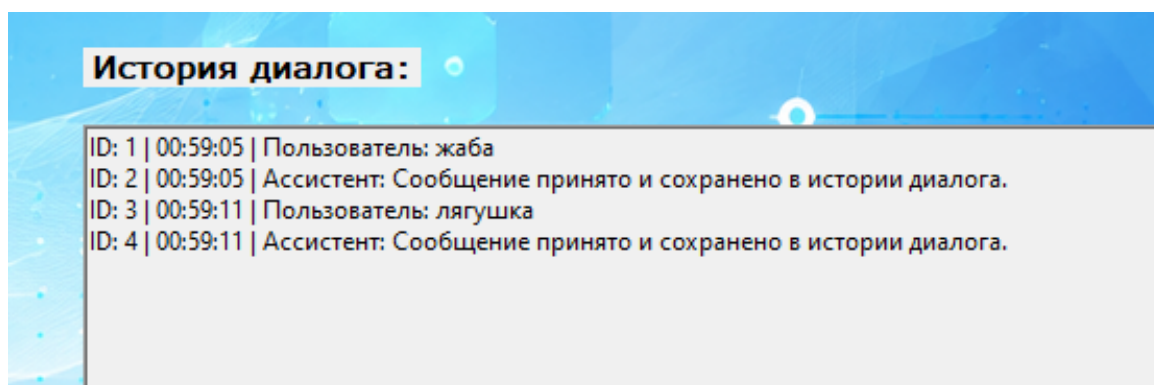


Рисунок 1 – Отображение истории диалога после добавления сообщений

На рисунке 1 показан результат работы алгоритма добавления сообщения в историю диалога. После ввода текста и нажатия кнопки отправки новая запись помещается в конец списка сообщений и сразу отображается в области истории. Каждая запись содержит порядковый идентификатор, время добавления, автора сообщения и текст. Такой способ вывода позволяет сохранить последовательность переписки и наглядно показать работу выбранной структуры данных.

Алгоритм добавления сообщения связан с основной структурой данных приложения – списком. Новая запись помещается в конец списка, поэтому последовательность сообщений сохраняется в том порядке, в котором пользователь взаимодействовал с приложением.

2.3. Описание пользовательской части приложения

2.3.1. Построение внешнего вида приложения

Графический интерфейс приложения состоит из одного главного рабочего окна, в котором размещены элементы для ввода сообщения, отображения истории диалога, поиска по содержимому и управления сохранёнными записями. Такое построение позволяет сосредоточить все основные действия пользователя в одной экранной форме.

Рабочую форму приложения можно условно разделить на несколько функциональных зон.

1. Область отображения истории диалога

Эта зона предназначена для вывода сохранённых сообщений пользователя и ассистента. Она занимает основную часть интерфейса, так как именно здесь отображается результат работы со списком истории.

- Поле истории диалога (HistoryTextBox) – основная область, в которой выводятся все сообщения. После добавления, удаления или очистки истории содержимое данного поля обновляется.
- Полоса прокрутки (ScrollBar) – элемент, позволяющий просматривать более ранние сообщения, если вся переписка не помещается в видимую область.
- Выделение найденного фрагмента (SelectionBackColor) – визуальный механизм, с помощью которого найденное слово подсвечивается светло-жёлтым цветом.

2. Панель ввода сообщения

Данная зона используется для добавления новых сообщений в историю. Пользователь вводит текст, после чего программа сохраняет его в общей последовательности переписки.

- Поле ввода сообщения (InputTextBox) – текстовое поле, в которое пользователь вводит новое сообщение.
- Кнопка отправки (ButtonSend) – элемент управления, запускающий добавление введённого текста в историю диалога.

3. Панель поиска по истории

Эта зона предназначена для поиска нужного слова или фразы в сохранённой переписке.

- Поле поиска (SearchTextBox) – строка, в которую пользователь вводит искомый фрагмент.
- Кнопка поиска (ButtonFind) – элемент управления, запускающий проверку истории на наличие введённого слова или фразы.

4. Панель управления историей

Данная зона отвечает за изменение содержимого истории диалога.

- Кнопка удаления последнего сообщения (ButtonDeleteLast) – удаляет последнюю добавленную запись из истории.
- Кнопка очистки истории (ButtonClear) – полностью очищает список сообщений и область отображения переписки.

5. Визуальное оформление формы

Внешний вид приложения дополняется декоративными и вспомогательными элементами, которые делают форму более завершённой.

- Фоновое изображение (BackgroundImage) – используется для визуального оформления главного окна приложения.
- Основная форма (MainForm) – главное окно программы, на котором размещаются все элементы интерфейса.

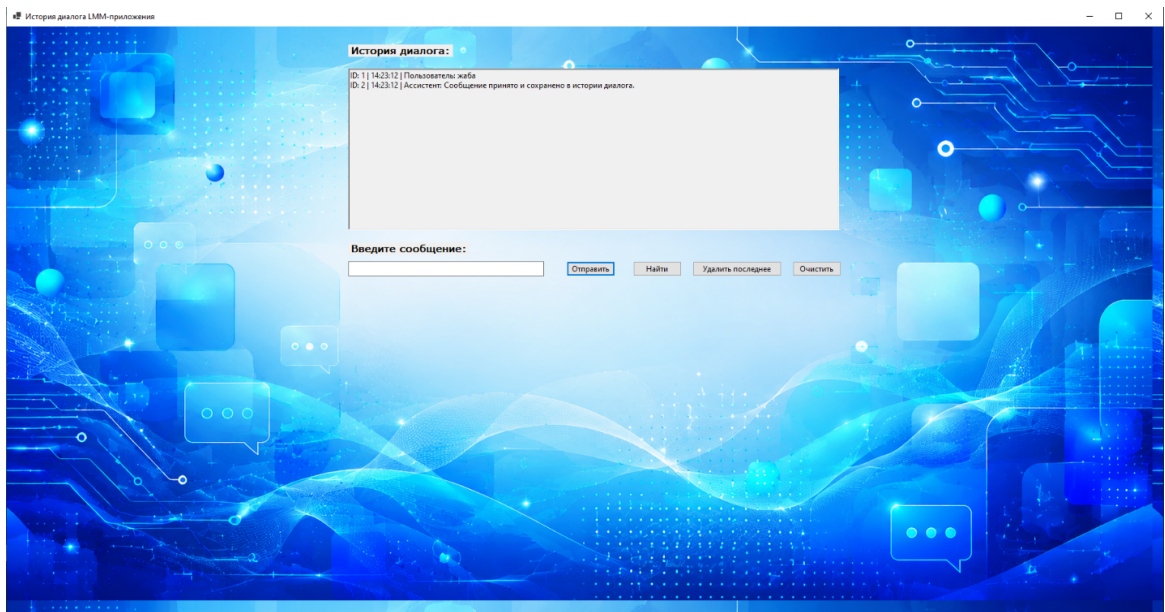


Рисунок 2 – Главная экранная форма приложения для хранения истории диалога

Таким образом, экранная форма приложения состоит из области истории, панели ввода, блока поиска и кнопок управления. Каждый элемент связан с определённым действием пользователя и участвует в обработке истории диалога.

На рисунке 2 представлена главная экранная форма разработанного приложения. В центральной части расположена область отображения истории диалога, в нижней части – поле ввода сообщения и элементы управления. Пользователь может добавить новое сообщение, выполнить поиск по сохранённой истории, удалить последнюю запись или полностью очистить переписку.

2.3.2. Последовательность работы пользователя с интерфейсом

После запуска приложения перед пользователем открывается основная форма, на которой расположены все элементы для работы с историей диалога. В верхней или центральной части формы находится область отображения истории, ниже располагается поле ввода сообщения и кнопки управления.

Последовательность работы пользователя с приложением:

Запуск приложения. Пользователь открывает программу, после чего на экране появляется главная форма (MainForm) с рабочими элементами интерфейса.

Ввод сообщения. Пользователь вводит текст в поле сообщения (InputTextBox). На этом этапе данные ещё не добавляются в историю, а только подготавливаются к отправке.

Отправка сообщения. После нажатия кнопки отправки (ButtonSend) программа считы-

вает введенный текст. Если поле не пустое, сообщение добавляется в историю и отображается в области переписки (HistoryTextBox).

Просмотр истории диалога. Все добавленные сообщения отображаются в области истории (HistoryTextBox) в порядке их добавления. При большом количестве записей пользователь может перемещаться по переписке с помощью полосы прокрутки (ScrollBar).

Поиск по истории. Для поиска нужного фрагмента пользователь вводит слово или фразу в поле поиска (SearchTextBox) и нажимает кнопку поиска (ButtonFind). При нахождении совпадения программа выделяет найденный фрагмент с помощью механизма подсветки (SelectionBackColor).

Удаление последнего сообщения. Если необходимо убрать последнюю добавленную запись, пользователь нажимает кнопку удаления (ButtonDeleteLast). После выполнения операции последнее сообщение удаляется из истории, а область отображения обновляется.

Очистка истории. При необходимости полного сброса переписки пользователь нажимает кнопку очистки (ButtonClear). После этого список сообщений очищается, а область истории становится пустой.

Продолжение работы или завершение приложения. После выполнения операций пользователь может продолжить ввод новых сообщений или закрыть программу.

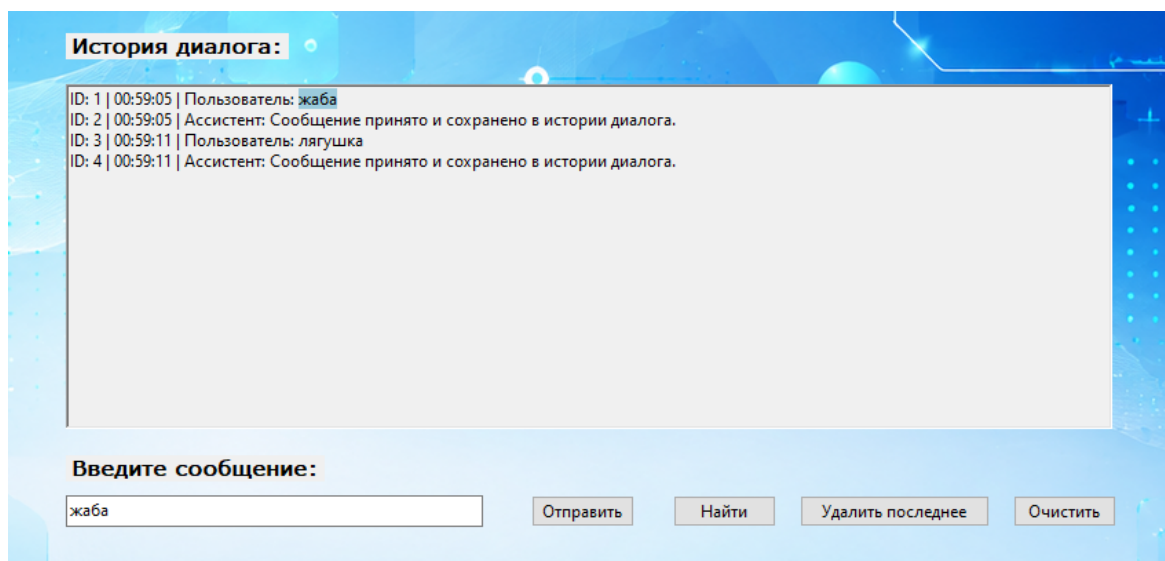


Рисунок 3 – Выделение найденного слова в истории диалога

На рисунке 3 показан результат работы функции поиска по истории диалога. Пользователь вводит искомое слово в поле ввода и нажимает кнопку поиска. После этого программа просматривает сохранённые сообщения и выделяет найденный фрагмент в области истории.

Такой сценарий показывает, каким образом пользователь взаимодействует с основными элементами приложения. Каждое действие связано с определённым элементом интерфейса и

приводит к изменению истории диалога или её отображения на экране.

2.4. Описание контрольного примера

2.4.1. Характеристика тестового набора сообщений

Для проверки работоспособности приложения используется небольшой набор текстовых сообщений, которые вводятся пользователем вручную через поле ввода. Такой вариант проверки позволяет проследить, как программа добавляет новые записи в историю, отображает их в области переписки, выполняет поиск по содержимому и удаляет последнее сообщение.

В качестве тестовых данных используются сообщения разной длины и содержания. Это необходимо для того, чтобы проверить работу приложения не только с короткими фразами, но и с обычными предложениями. Все сообщения добавляются последовательно и сохраняются в истории в том порядке, в котором пользователь вводит их в программу.

Таблица 1 – Тестовый набор сообщений для проверки приложения

№	Тестовое сообщение	Назначение проверки
1	Сегодня нужно подготовить отчёт	Проверка добавления первого сообщения в историю
2	Напомнить проверить расписание занятий	Проверка сохранения обычного текстового сообщения
3	Купить продукты после университета	Проверка добавления сообщения другой тематики
4	Отчёт нужно отправить до вечера	Проверка повторяющегося слова для поиска
5	Последнюю запись нужно удалить	Проверка удаления последнего сообщения

После ввода данных сообщений в области истории должна появиться последовательность записей. Каждая новая строка добавляется в конец истории, поэтому порядок сообщений соответствует порядку их ввода. Это позволяет проверить, что выбранная структура данных корректно сохраняет последовательность элементов.

Отдельно проверяется функция поиска. Для этого в качестве искомого слова используется слово «отчёт», так как оно встречается в первом и четвёртом сообщении тестового набора. После запуска поиска программа должна найти совпадение в тексте истории и выделить найденный фрагмент светло-жёлтым цветом.

Также данный набор сообщений позволяет проверить удаление последнего элемента истории. После нажатия кнопки удаления из области истории должна исчезнуть последняя добавленная запись: «Последнюю запись нужно удалить». Остальные сообщения при этом должны

остаться без изменений, так как операция удаления применяется только к последнему сообщению.

Таким образом, тестовый набор сообщений позволяет проверить основные функции приложения на простом контрольном примере. Он показывает работу добавления, отображения, поиска и удаления сообщений без использования внешних файлов или базы данных.

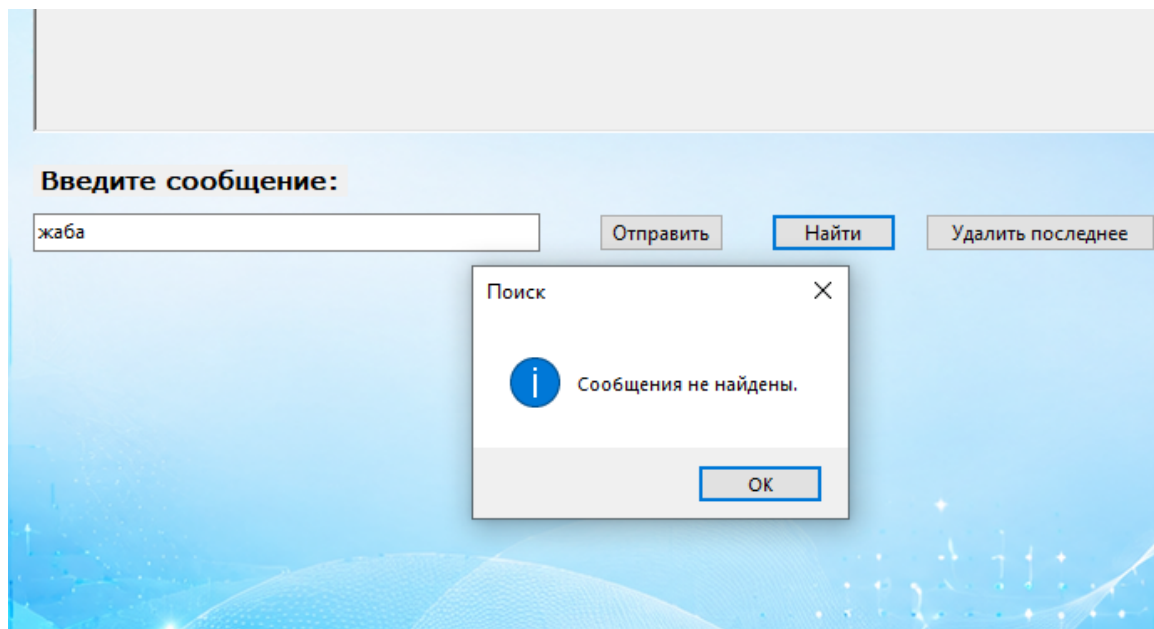


Рисунок 4 – Сообщение о результате поиска при отсутствии совпадений

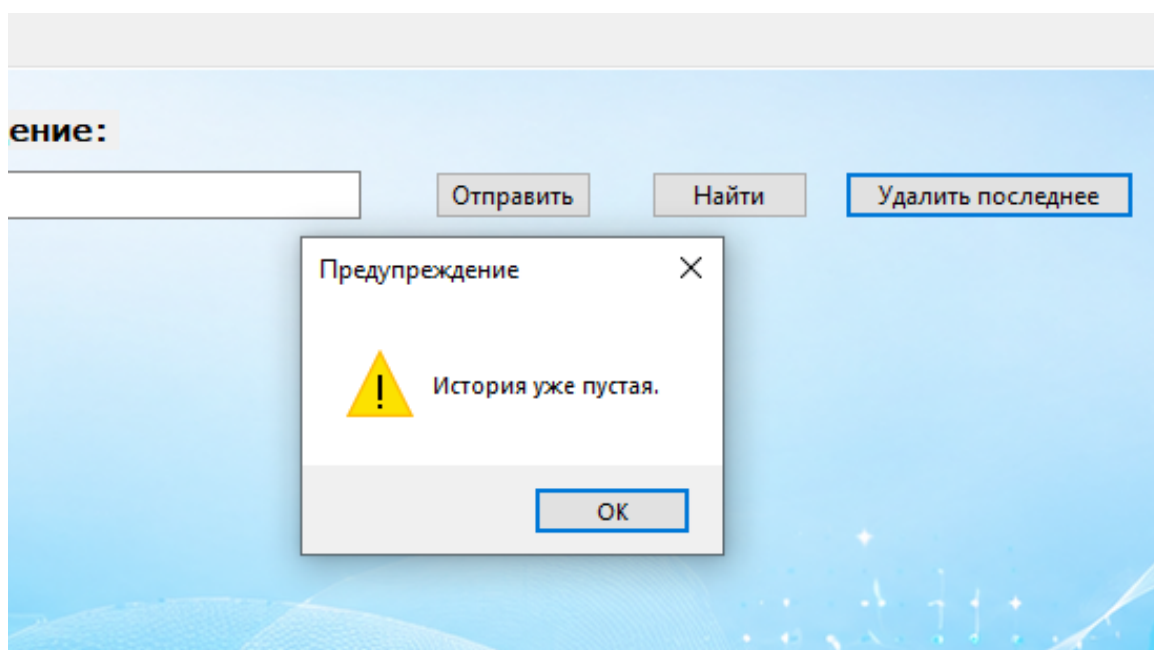


Рисунок 5 – Предупреждение при попытке удалить сообщение из пустой истории

На рисунках 4 и 5 показана обработка нестандартных ситуаций при работе с приложением. Если пользователь пытается удалить сообщение при пустой истории, программа выводит

предупреждение. При поиске слова, которого нет в сохранённых сообщениях, приложение также выводит информационное сообщение о том, что совпадения не найдены.

Заключение

В ходе выполнения курсового проекта было разработано приложение для хранения и обработки истории диалога. Основное внимание в работе было уделено проектированию структуры данных, которая позволяет сохранять сообщения пользователя, отображать их в правильной последовательности и выполнять основные операции над историей.

В теоретической части были рассмотрены особенности LMM-приложений и роль истории диалога в подобных системах. Было установлено, что история сообщений является важной частью диалогового взаимодействия, так как она позволяет сохранять последовательность общения и использовать ранее введенные данные. Также были изучены основные структуры данных, которые могут применяться для хранения сообщений, включая список, очередь, словарь, дерево и граф.

Для реализации приложения в качестве основной структуры данных был выбран список. Данный выбор обусловлен тем, что история диалога имеет линейный характер: новые сообщения добавляются последовательно, а при отображении должны сохранять порядок ввода. Список позволяет удобно добавлять новые элементы, просматривать историю, выполнять поиск по содержимому и удалять последнюю запись.

В практической части было спроектировано и реализовано приложение с графическим интерфейсом. В программе предусмотрены поле ввода сообщения, область отображения истории, кнопка добавления записи, кнопка поиска, кнопка удаления последнего сообщения и кнопка очистки истории. Пользователь может вводить сообщения, сохранять их в общей истории, находить нужные слова в тексте и управлять содержимым переписки.

Отдельное внимание было уделено алгоритмам обработки данных. Были описаны алгоритмы добавления сообщения в историю, поиска по сохраненному тексту, удаления последней записи и очистки всей истории. На контрольном наборе сообщений была проверена работа основных функций приложения. В результате проверки было установлено, что программа корректно добавляет сообщения, отображает их в области истории, выполняет поиск с выделением найденного фрагмента и удаляет последнее сообщение без изменения остальных записей.

Таким образом, цель курсового проекта была достигнута. Было создано простое приложение, демонстрирующее применение структур данных для хранения истории диалога. Разработанное решение может служить основой для дальнейшего расширения: добавления сохранения истории в файл, подключения базы данных, поддержки нескольких диалогов, улучшения поиска и интеграции с более сложными диалоговыми системами.

Библиографический список

1. Вирт, Н. Алгоритмы и структуры данных / Н. Вирт. – Москва : Мир, 1989. – 360 с.
2. Кормен, Т. Х. Алгоритмы: построение и анализ / Т. Х. Кормен, Ч. И. Лейзерсон, Р. Л. Ривест, К. Штайн. – 3-е изд. – Москва : Вильямс, 2013. – 1328 с.
3. Шихи, Д. Р. Структуры данных в Python: начальный курс / Д. Р. Шихи ; пер. с англ. А. В. Снастина. – Москва : ДМК Пресс, 2022. – 186 с.
4. Гудрич, М. Структуры данных и алгоритмы в Python / М. Гудрич, Р. Тамассия, М. Голдвасер. – Москва : ДМК Пресс, 2019. – 740 с.
5. Лутц, М. Изучаем Python. Том 1 / М. Лутц. – 5-е изд. – Санкт-Петербург : Диалектика, 2019. – 832 с.
6. Васильев, А. Н. Программирование на C# для начинающих и профессионалов / А. Н. Васильев. – Москва : Эксмо, 2022. – 704 с.
7. Павловская, Т. А. C#. Программирование на языке высокого уровня / Т. А. Павловская. – Санкт-Петербург : Питер, 2020. – 432 с.
8. Шилдт, Г. C# 4.0: полное руководство / Г. Шилдт. – Москва : Вильямс, 2011. – 1056 с.
9. Троелсен, Э. Язык программирования C# и платформа .NET / Э. Троелсен. – Санкт-Петербург : Питер, 2018. – 1328 с.
10. Макконнелл, С. Совершенный код. Мастер-класс / С. Макконнелл. – Москва : Русская редакция, 2017. – 896 с.
11. Фаулер, М. Рефакторинг. Улучшение существующего кода / М. Фаулер. – Санкт-Петербург : Питер, 2019. – 448 с.
12. Рассел, С. Искусственный интеллект: современный подход / С. Рассел, П. Норvig. – Москва : Вильямс, 2020. – 1408 с.
13. Николенко, С. И. Глубокое обучение / С. И. Николенко, А. А. Кадури, Е. О. Архангельская. – Санкт-Петербург : Питер, 2018. – 480 с.
14. Хайкин, С. Нейронные сети: полный курс / С. Хайкин. – Москва : Вильямс, 2019. – 1104 с.

15. Microsoft Learn. Документация по Windows Forms и элементам управления [Электронный ресурс]. – Режим доступа: <https://learn.microsoft.com/ru-ru/dotnet/desktop/winforms/>

Приложение А. Код программы

```

using System;
using System.Collections.Generic;
using System.Drawing;
using System.Linq;
using System.Windows.Forms;

namespace Курсовая
{
    public partial class Form1 : Form
    {
        private List<DialogMessage> dialogHistory = new List<DialogMessage>();
        private Stack<DialogMessage> deletedMessages = new Stack<DialogMessage>();
        private Queue<DialogMessage> messageQueue = new Queue<DialogMessage>();
        private Dictionary<int, DialogMessage> messageDictionary = new
        Dictionary<int, DialogMessage>();

        private int messageId = 1;

        public Form1()
        {
            InitializeComponent();
            this.StartPosition = FormStartPosition.CenterScreen;
            this.Text = "История диалога LMMприложения-";
        }

        private void AddMessage(string role, string text)
        {
            DialogMessage message = new DialogMessage(messageId, role, text,
            DateTime.Now);

            dialogHistory.Add(message);
            messageQueue.Enqueue(message);
            messageDictionary.Add(message.Id, message);

            messageId++;
        }

        private void UpdateHistory()
        {
            txtHistory.Clear();

            foreach (DialogMessage message in dialogHistory)
            {
                txtHistory.AppendText(
                    $"ID: {message.Id} | {message.Time:HH:mm:ss} | " +
                    $"{message.Role}: {message.Text}{Environment.NewLine}"
                );
            }

            txtHistory.SelectionStart = txtHistory.Text.Length;
            txtHistory.ScrollToCaret();
        }

        private void SendMessage()
        {
            string userText = txtInput.Text.Trim();

```

```

    if (userText == "")
    {
        MessageBox.Show(
            this,
            "Введите сообщение.",
            "Предупреждение",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning
        );
        return;
    }

    AddMessage("Пользователь", userText);

    string botAnswer = "Сообщение принято и сохранено в истории диалога.";
    AddMessage("Ассистент", botAnswer);

    txtInput.Clear();
    UpdateHistory();
}

private void SearchMessage()
{
    string searchText = txtInput.Text.Trim();

    if (searchText == "")
    {
        MessageBox.Show(
            this,
            "Введите слово для поиска.",
            "Предупреждение",
            MessageBoxButtons.OK,
            MessageBoxIcon.Warning
        );
        return;
    }

    UpdateHistory();

    string historyText = txtHistory.Text;
    int index = historyText.IndexOf(searchText,
StringComparison.CurrentCultureIgnoreCase);

    if (index == -1)
    {
        MessageBox.Show(
            this,
            "Сообщения не найдены.",
            "Поиск",
            MessageBoxButtons.OK,
            MessageBoxIcon.Information
        );
        return;
    }

    txtHistory.Select(index, searchText.Length);
    txtHistory.SelectionBackColor = Color.LightYellow;
    txtHistory.SelectionColor = Color.Black;
    txtHistory.ScrollToCaret();
}

```

```

        txtHistory.Focus();
    }

    private void DeleteLastMessage()
    {
        if (dialogHistory.Count == 0)
        {
            MessageBox.Show(
                this,
                "История уже пустая.",
                "Предупреждение",
                MessageBoxButtons.OK,
                MessageBoxIcon.Warning
            );
            return;
        }

        DialogMessage lastMessage = dialogHistory[dialogHistory.Count - 1];
        dialogHistory.RemoveAt(dialogHistory.Count - 1);
        deletedMessages.Push(lastMessage);
        messageDictionary.Remove(lastMessage.Id);

        if (dialogHistory.Count > 0)
        {
            DialogMessage previousMessage = dialogHistory[dialogHistory.Count
- 1];
            dialogHistory.RemoveAt(dialogHistory.Count - 1);
            deletedMessages.Push(previousMessage);
            messageDictionary.Remove(previousMessage.Id);
        }

        UpdateHistory();
    }

    private void ClearHistory()
    {
        dialogHistory.Clear();
        deletedMessages.Clear();
        messageQueue.Clear();
        messageDictionary.Clear();

        txtHistory.Clear();
        txtInput.Clear();

        messageId = 1;
    }

    private void btnSend_Click(object sender, EventArgs e)
    {
        SendMessage();
    }

    private void btnSearch_Click(object sender, EventArgs e)
    {
        SearchMessage();
    }

    private void btnUndo_Click(object sender, EventArgs e)
    {

```

```

        DeleteLastMessage();
    }

    private void btnClear_Click(object sender, EventArgs e)
    {
        ClearHistory();
    }

    private void btnSend_Click_1(object sender, EventArgs e)
    {
        SendMessage();
    }

    private void btnSearch_Click_1(object sender, EventArgs e)
    {
        SearchMessage();
    }

    private void btnUndo_Click_1(object sender, EventArgs e)
    {
        DeleteLastMessage();
    }

    private void btnClear_Click_1(object sender, EventArgs e)
    {
        ClearHistory();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        SendMessage();
    }

    private void button2_Click(object sender, EventArgs e)
    {
        SearchMessage();
    }

    private void button3_Click(object sender, EventArgs e)
    {
        DeleteLastMessage();
    }

    private void button4_Click(object sender, EventArgs e)
    {
        ClearHistory();
    }
}

public class DialogMessage
{
    public int Id { get; set; }
    public string Role { get; set; }
    public string Text { get; set; }
    public DateTime Time { get; set; }

    public DialogMessage(int id, string role, string text, DateTime time)
    {
        Id = id;
    }
}

```

```
1         Role = role;  
2         Text = text;  
3         Time = time;  
4     }  
5 }  
6 }
```